

杭电春季联赛第六场题解

济南大学

LUA 出题组

2026 年 4 月 24 日

A 飞燕草

题目描述：加边使得森林变成树，最小化直径，新增边权之和至少为 S 。

结论 1：两棵树连接到一起得到的树的直径，两个端点一定来源于原本两棵树直径的 4 个点。

所以我们只需要把每个连通块当作一条链，在所有链的中点相互连边。

结论 2：将距离链中点最远的点，与中点的距离称作链的半径，最优方案一定是半径最大的链中点连接所有其他中点。

A 飞燕草

此时可以二分最终树的直径，然后看新增边权之和最大值是多少。

将链半径从大到小排序，第 i 条链半径为 r_i ，和 1 号链连边的长度为 d_i ，我们要保证 $r_1 + r_i + d_i$ 和 $r_i + r_j + d_i + d_j$ 不超过二分阈值，贪心的角度希望 $r_i + d_i$ 尽可能均衡，我们给 r_i 小的直径设置更长的边。

时间复杂度 $O(n \log n)$ ，有更好的实现可以做到 $O(n)$ 。

B 滚动的色彩

题目描述：给定 $n \times m$ 的网格，一个标记有颜色的正方体骰子在网格上滚动。求从起点 (sx, sy) 滚动到终点 (tx, ty) 且最终底面颜色恰好为 ct 的最小滚动次数。

这道题本质上是在无权图上求最短路的问题，由于每次滚动的代价都是 1，我们可以直接使用 BFS 来求解。

难点在于如何记录和转移骰子的状态。除了骰子当前所在的坐标 (x, y) ，仅仅记录坐标是不够的，因为到达同一个格子时骰子的朝向可能不同，从而影响后续的滚动。我们需要将骰子当前六个面（顶、前、右、左、后、底）的颜色分布也加入状态中。因为一个标准正方体在三维空间中只有 24 种不同的旋转姿态，所以同一个网格坐标最多只会对应 24 种不同的有效颜色分布状态。

我们将状态表示为 $(x, y, c_1, c_2, c_3, c_4, c_5, c_6)$ 。当骰子向四个方向（上、下、左、右）滚动时，不仅坐标 (x, y) 会发生相应的改变，面上颜色的相对位置也会按照固定的置

B 滚动的色彩

换规则改变。通过预处理好四个方向的颜色置换数组，就可以在 BFS 扩展节点时 $O(1)$ 获得下一步的六个面颜色分布。

使用三维状态数组或 `std::map<array<int, 6>, int> mp[N][N]` 记录某个坐标下某种颜色分布是否已经被访问过，防止重复搜索。在 BFS 过程中，一旦弹出的队首元素满足当前坐标等于 (tx, ty) 且底面颜色 c_6 等于目标颜色 ct 时，记录的滚动步数即为所求的最小步数。如果队列遍历完毕仍未找到，则输出 -1 。

时间复杂度为 $O(nm)$ 。

C 绩点查询

题目描述：根据题面给出的同时计算绩点。

直接套用题目中给出的计算公式即可，注意乘权重相加向上取整时可能出现精度问题，可以让权重乘 10 变成整数，完全规避精度问题。

D 再次出发，前往地球

题目描述：给定一个凸包，多次询问两个凸包外的点不穿过凸包的最短路，点数和询问数 10^5 。

首先判断 ST 是否与凸包相较：做两条与 ST 平行的凸包的切线，判断 ST 是否在两条切线之间，如果在，求出直线 ST 与凸包的交点 A, B ，判断四个点的位置关系即可判断是否与凸包相较。

如果相较，分别从 S, T 做凸包的两条切线，得到四个切点，最优路线一定是从 S 走到一个切点，沿着凸包走到另一个切点，在走到 T 。

如何做切线？这里提供一种思路：过 S 做一条直线切割凸包，两个切点分别在两侧，可以二分得到切点。

总时间复杂度 $O(n\log^2 n)$ 或 $O(n\log n)$ 。

E 向量压缩

题目描述：给定长度为 n 的序列，当出现 k 个相邻且相同的元素时，它们会发生“湮灭”并消失，剩余的元素会重新拼接并可能引发连锁反应。求最终稳定的序列。

从左到右遍历序列，用栈维护元素及其连续出现的次数。如果当前元素与栈顶元素相同，则栈顶计数加一；如果不同或栈为空，则将当前元素及初始计数 1 压入栈。每次计数更新后检查是否达到 k ，若达到则将计数减 k （触发消除）。最后将栈中剩余元素按计数展开并反转输出即可。

时间复杂度为 $O(n)$ 。

F 括号路径

题目描述：给定无向图，边上写着左括号或者右括号，多次询问两点间最短路，路径上字符按顺序记录下来是括号串，点数 400，边数 800。

如何判定一个字符串是否是括号串：从左往右扫，记录未匹配的左括号数量 x ，遇到左括号让 x 加一，右括号让 x 减一，如果最终 $x = 0$ 其过程中没有变成过负数，那这就是一个合法括号串。

以每个点作为源点跑 Dijkstra 算法，记录 $d_{i,j}$ 表示从起点到 i ，路径上剩余未匹配的左括号数量为 j 的最短路， $d_{i,0}$ 就是起点到 i 的最短括号路径。

时间复杂度 $O(n^2m + q)$ 。

G 不开心世纪

题目描述：给定序列，问最少删除多少元素，使得删除后的序列存在长度为 m 的排列。

考虑最终排列出现在区间 $[l, r]$ 内，那我们删除区间外部的元素没有意义，二分答案 x ，枚举所有长度为 $m + x$ 的子段，判断子段内是否有 $1, 2, \dots, m$ 所有数字，时间复杂度 $O(n \log n)$ 。

也可以使用双指针，右指针右移时维护最靠右的左指针，使得区间内有 $1, 2, \dots, m$ 所有数字，时间复杂度 $O(n)$ 。

H 黑白树

题目描述：给定无向图，每条边是黑色或者白色，问有多少二元组 (x, y) ，原图存在 x 条黑边， y 条白边的生成树，点数 10^5 。

由于边数总和 $n - 1$ 固定，确定了一种颜色的边数量后，另一种颜色的边数也就确定了。

不妨考虑白边数量可能的取值，不难发现取值范围一定连续，只需要求出取值范围的上下界即可。

考虑将白边边权设成 1，黑边边权设成 0，此时最小生成树对应的边权和即为构成生成树所需的白边数量下界 L 。同理，最大生成树对应的边权和即为白边数量上界 R 。

最后 $R - L + 1$ 即为答案。注意特判不能构成生成树的情况。

I 月之都

题目描述：给定 n 个点的完全无向图，任意两点 i, j 的边权为 $i \oplus j$ 。额外给定 m 条带权无向边，求从 1 到 n 经过边权的异或和最小的路径。

结论 1：如果只走原图的隐式边，从 1 到 n 无论经过哪些点，中间节点在连续异或中都会两两抵消（例如 $1 \oplus u \oplus u \oplus v \oplus v \oplus n = 1 \oplus n$ ），因此纯隐式边路径的异或和恒定为 $1 \oplus n$ 。

结论 2：每借用一条额外边 (u, v, w) ，等价于在原路径上额外绕了一个环，由于进出该环的隐式边也会抵消，这个环对总路径产生的异或贡献固定为 $u \oplus v \oplus w$ 。这是因为从任意点 x 经额外边 (u, v, w) 走到点 y 的代价为 $(x \oplus u) \oplus w \oplus (v \oplus y) = (x \oplus y) \oplus (u \oplus v \oplus w)$ ，即借用额外边，便相当于在原权值上额外异或了 $u \oplus v \oplus w$ 。

因此，任意一条从 1 到 n 的路径异或和，等价于初始值 $1 \oplus n$ 异或上若干个额外边构成的环权值。我们只需将 m 条额外边对应的 $u \oplus v \oplus w$ 全部插入线性基，然后以 $1 \oplus n$ 为起点，在线性基中贪心求最小值即可。

时间复杂度 $O(m \log V)$ 。

题目描述：给定长度为 n 的序列，问有多少长度为 k 的子序列，元素异或和为 0， $n \leq 10^6, k \leq 8$ 。

原序列数字记在桶 T_A 中，选择两个相同位置的数字的异或和记在桶 T_{AA} 中（实际上是 n 个 0），接下来对桶做的乘法是异或卷积（快速沃尔什变换）。

$k = 2$ 时， $T_{AB} = T_A T_A - T_{AA}$ ，得到的桶 T_{AB} 表示选择两个不同位置得到的异或和，答案为 $\frac{T_{AB}[0]}{2}$ 。

$k = 3$ 时， $T_{AAB} = T_{AA} T_A - T_{AAA}$ ， $T_{ABC} = T_{AB} T_A - 2T_{AAB}$ ，其中 T_{AAA} 表示选择三个相同位置， T_{AAB} 表示选择三个位置，其中两个位置相同， T_{ABC} 表示选择三个不同位置，即为长度为三的子序列，答案为 $\frac{T_{ABC}[0]}{6}$ 。

k 更大时, 组合情况会变得复杂, 组合形式的数量为 k 的整数划分数 (五边形数), 如果讨论所有形式的计算, 代码会变得非常复杂, 此时可以直接搜状态, 简化代码, 答案为 $\frac{T_{ABC...X}[0]}{k!}$ 。

时间复杂度为上界为 $O(2^k n \log n)$, 实际远低于上界, 大致为 $O(P(k) n \log n)$ ($P(n)$ 表示将 n 划分为若干正整数之和, 正整数可重集的数量)。

提供一个效率更高的方法, $T_{AABC...X}$ 可以直接由 $kT_{ABC...X}$ 得到, k 是序列长度减去 $ABC...X$ 的长度, 时间复杂度为 $O(nk \log n)$ 。

致谢

Thanks !